Föreläsning 2

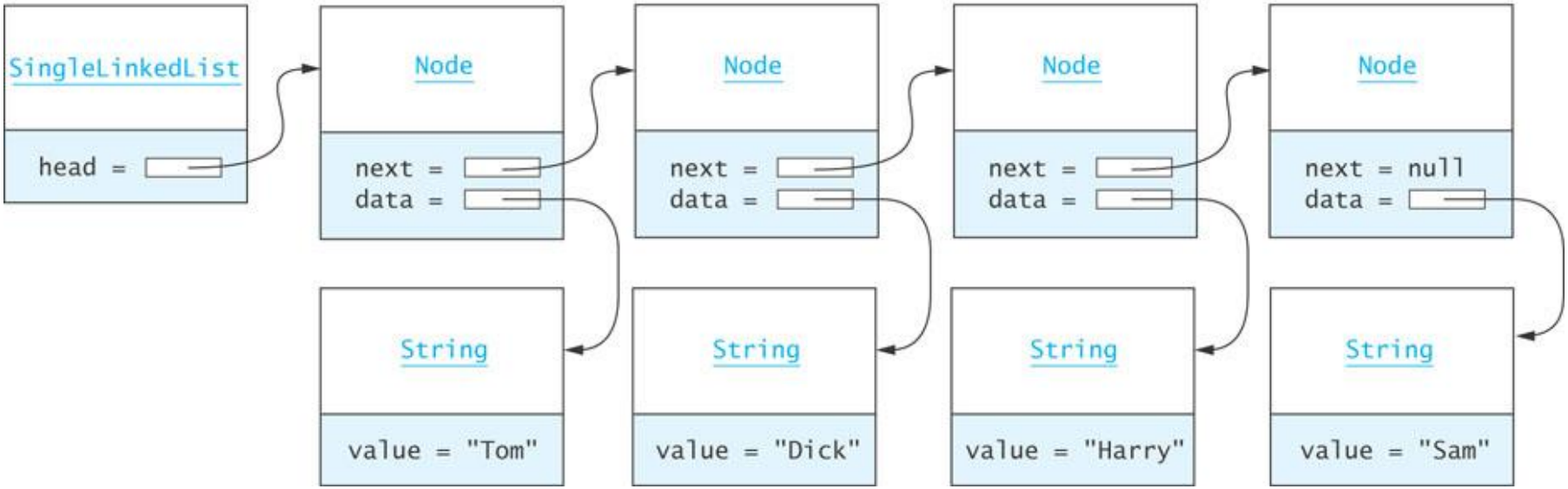
Länkad lista och iterator

Föreläsning 2

- Länkad-lista
- Lista implementerad med en enkellänkad lista
- Iterator
- Implementering av en Iterator
- Dubbellänkad lista och cirkulär lista
- LinkedList JCF
- Iterator JCF

Länkad lista (enkellänkad)

- En **länkad** lista är en datastruktur där varje element (data) ligger i en **nod** som håller reda på nästa nod.

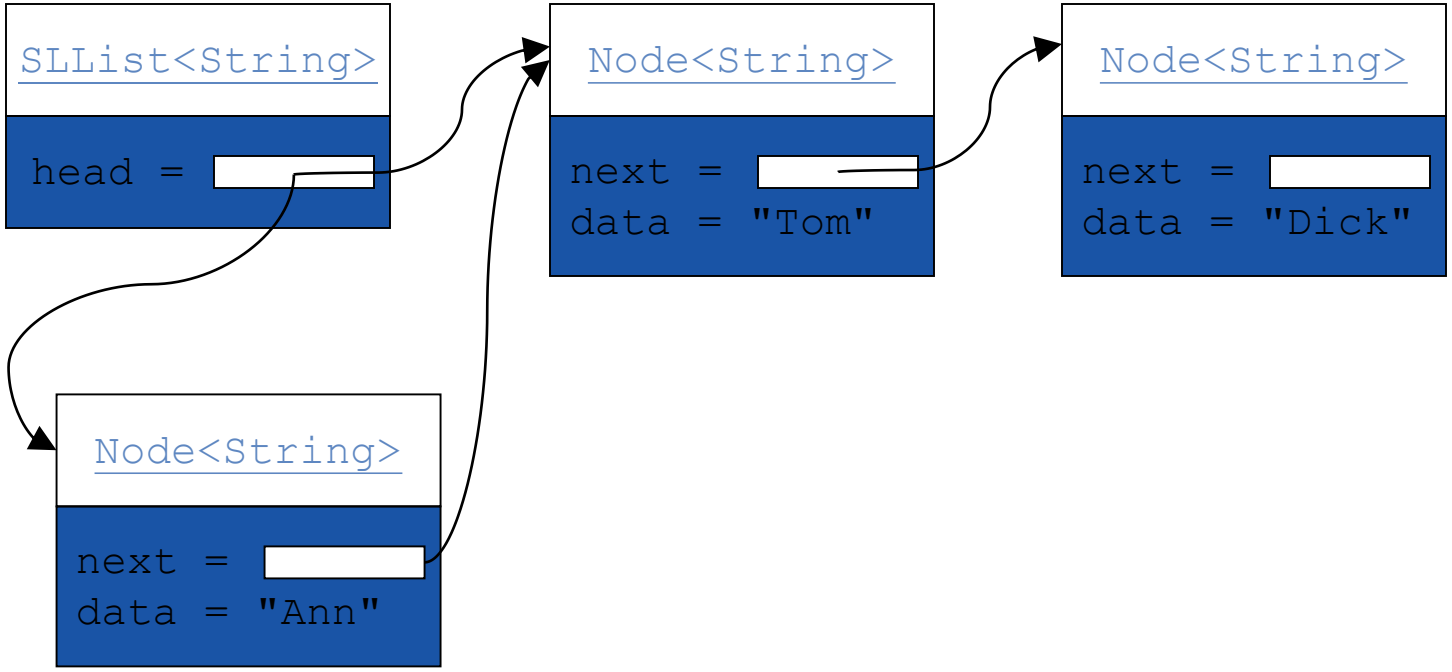


Observera att det endast är referenser (pekare) som ligger i listan. Objekten (i detta fall strängarna) ligger på heapen!

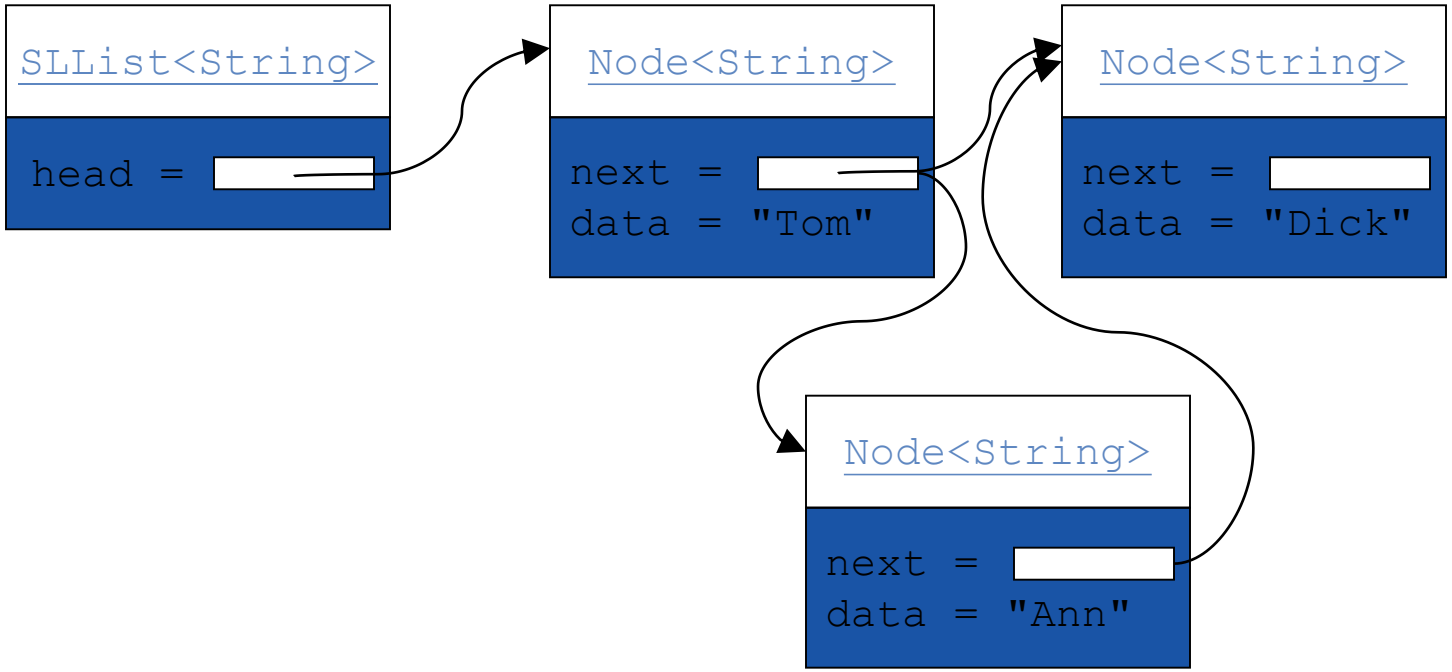
Implementering av lista med länkad lista jämfört med implementering med array

- Fördelarna
  - den kan växa dynamiskt
  - det är effektivt att stoppa in eller ta ut element mitt i. Har vi referensen till noden innan är att ta bort eller sätta in element O(1). Gäller då tex att sätta in något först.
- Nackdelarna
  - Varje element kräver plats för en referens till nästa nod
  - Att ta fram ett element på en viss plats är O(n)

Lägga till element först



Lägga till element mitt i listan



Observera att vi måste ha en referens till noden före för att kunna ändra dess pekare

Implementering av en lista med en länkad lista

Gränssnittet utåt för denna klass skall vara identiskt med vår arraylista.

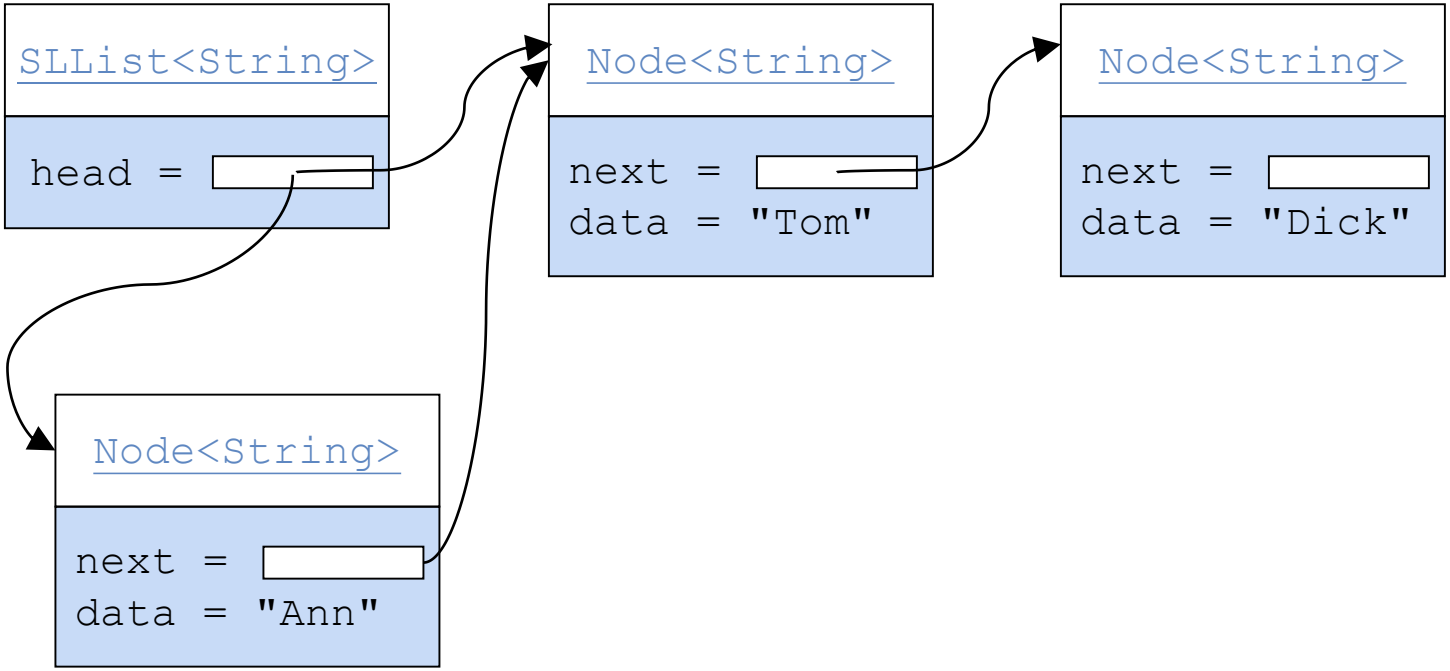
När man använder klassen ska man alltså inte behöva veta något om noder och länkar utan allt detta ska hanteras internt av klassen.

Kod

```
package singleLp;

public class SingleLinkedList<E> {

  private static class Node<E> {

     private E data;
     private Node<E> next;

     public Node(E data, Node<E> next) {
         this.data = data;
         this.next = next;
      }

   }

   private Node<E> head;
   private int size;

   public SingleLinkedList(){
      head = null;
      size = 0;
   }
…
```

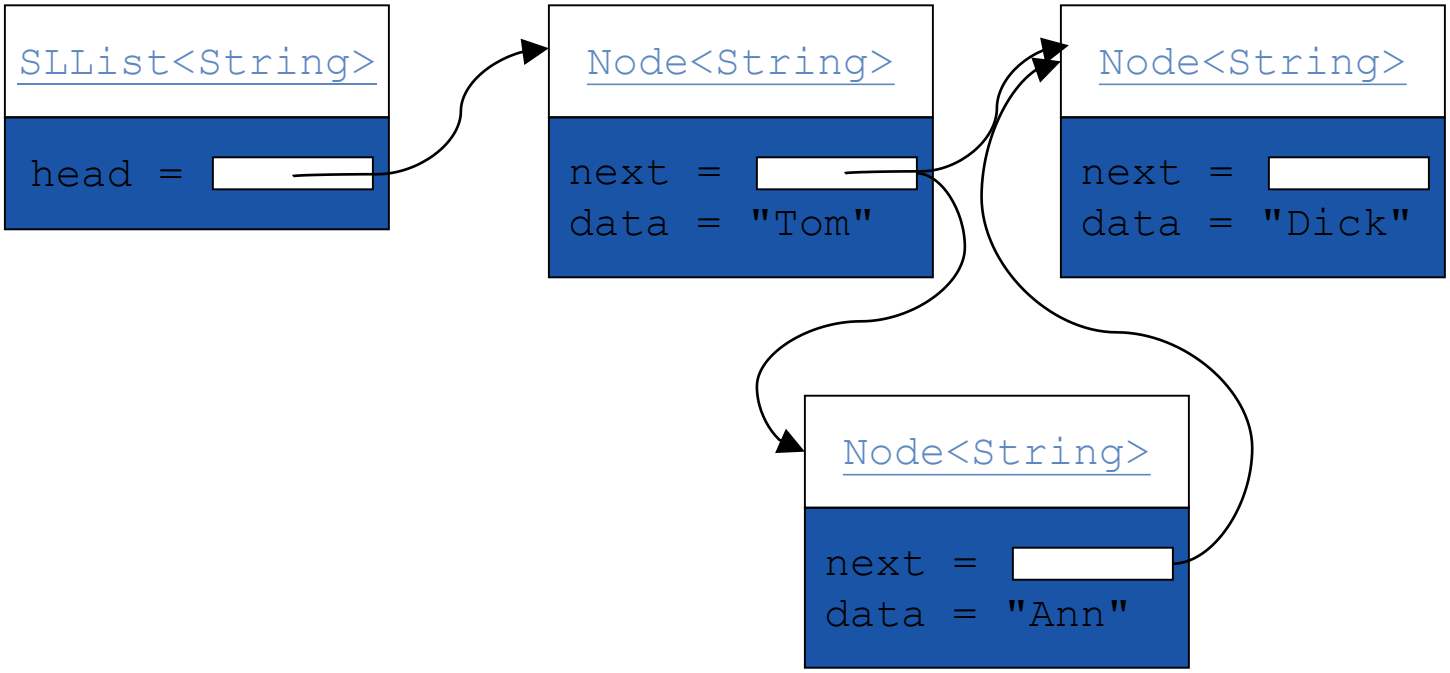
```
public void add(int index, E item) {
    if (index < 0 || index > size) {
        throw new IndexOutOfBoundsException(Integer.toString(index));
    }
    if (index == 0) {
        addFirst(item);
    } else {
        Node<E> node = getNode(index - 1);
        addAfter(node, item);
    }
}
```

```
public void addFirst(E item) {
    head = new Node<E>(item, head);
    size++;
}
```



```
//Returnerar null om noden saknas
private Node<E> getNode(int index) {
    Node<E> node = head;
    for (int i = 0; i < index && node != null; i++) {
        node = node.next;
    }
    return node;
}
```

```
private void addAfter(Node<E> node, E item) {
    node.next = new Node<E>(item, node.next);
    size++;
}
```



Lägg till element sist

- Det är vanligt att fylla på en lista genom att sätta elementen sist.
- Det krävs också av List-interfacet.

```
public boolean add(E item) {
    add(size, item);
    return true;
}
```

- När man implementerar en lista med en array är det mycket effektivt att lägga saker sist.
- Hur effektiv blir detta i en länkad lista?
- Vad händer om vi lägger till 10 element med denna till en tom lista?
- Hur kan vi förbättra effektiviteten?

```
public E get(int index) {
    if (index < 0 || index >= size) {
        throw new IndexOutOfBoundsException(Integer.toString(index));
    }
    Node<E> node = getNode(index);
    return node.data;
}
```

Även här får vi effektivitetsproblem om vi använder en for-loop och denna för att gå igenom en lista.

Traversera en lista

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder("[");
    Node<E> p = head;
    if (p != null) {
        while (p.next != null) {
            sb.append(p.data.toString());
            sb.append(" ==> ");
            p = p.next;
        }
        sb.append(p.data.toString());
    }
    sb.append("]");
    return sb.toString();
}
```

Här ser man hur man går igenom en lista utan att behöva gå igenom listan igen för varje element. En ide vi behöver ta med oss för att kunna gå igenom en lista effektivt även ”utifrån”.

Iterator

- En iterator låter oss gå igenom en listas element med O(1) per element
- Utan iterator får vi O(n) per element och vill vi gå igenom hela blir det $O(n^2)$
- Iteratorn åstadkommer detta genom att hålla reda på var i listan vi är
- På så sätt kan vi gå igenom en länkad lista lika effektivt som toString men utan att behöva veta något om noder

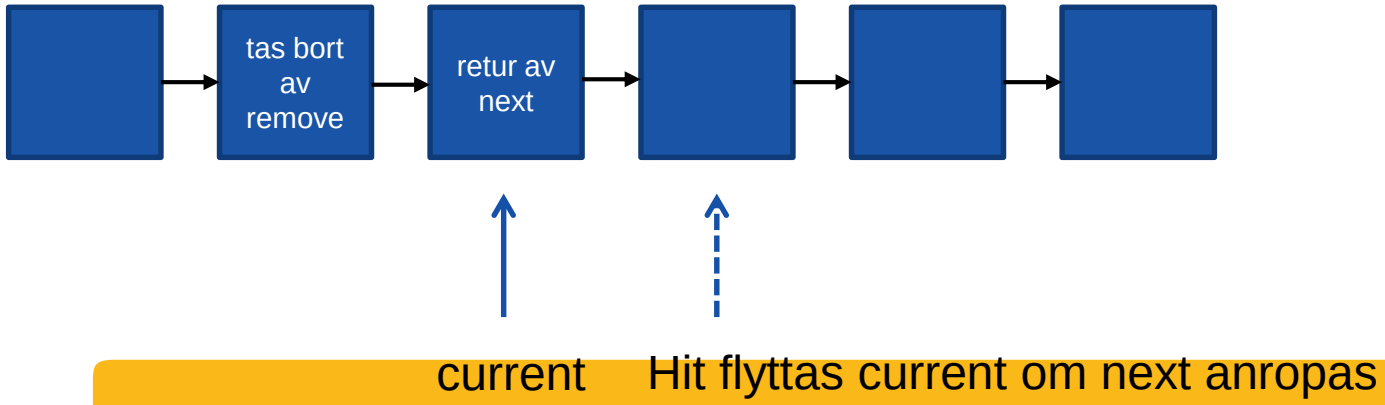
Iterator - state

Vid ett givet tillfälle har en iterator en pekare (current) som pekar på en av noderna (eller null) i listan.

hasNext() returnerar current!=null

next() returnerar current.data och flyttar fram current

remove() tar bort det senast av next() returnerade elementet.



Iterator - implementering

```
private class Itr implements Iterator<E> {//nested class – ligger i SingleLinkedList
    Node<E> current;

    public Itr(Node<E> start) {
        current = start;
    }
    @Override
    public boolean hasNext() {
        return current != null;
    }

    @Override
    public E next() {
        if (current == null) {
            throw new NoSuchElementException();
        }
        E returnValue = current.data;
        current = current.next;
        return returnValue;
    }
    @Override
    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

```
//metod i SingleLinkedList<E>:

public Iterator<E> iterator() {
        return new Itr(head);
}
```

// lägg också till att klassen **implements Iterable<E>**

Exempel iterator

```
package singleLp;

import java.util.Iterator;

public class SingleLinkedListTest {

    public static void main(String[] args) {
        SingleLinkedList<String> list = new SingleLinkedList<String>();
        for(int i=1;i<=10;i++)//O(n²)
            list.add("Sträng "+i);
        Iterator<String> iter = list.iterator();
        while(iter.hasNext())//O(n)
            System.out.println(iter.next());
    }
}
```

Dubbellänkad lista

I en dubbellänkad lista har varje nod en referens till både noden före och noden efter

Dessutom finns medlemsvariablerna head och tail

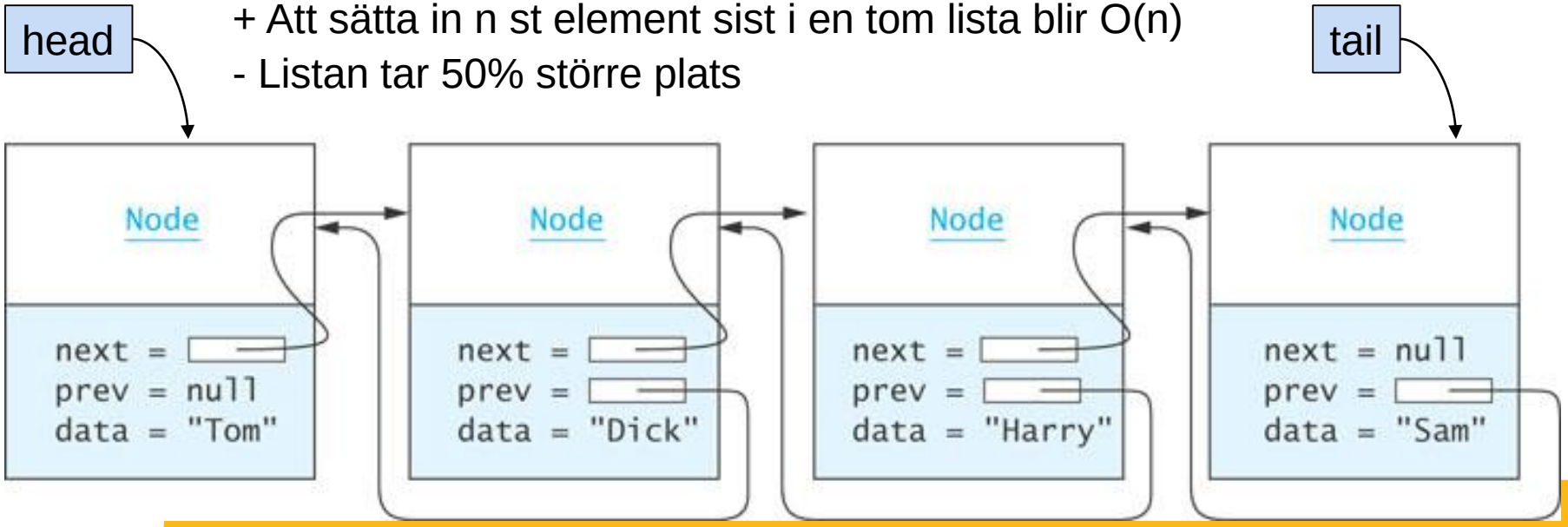
+ Vi kan sätta in före och efter en nod vi har en referens till

+ Vi kan ta bort en nod utan att ha en referens till noden innan

+ Vi kan traversera listan åt bägge håll

+ Att sätta in n st element sist i en tom lista blir O(n)

- Listan tar 50% större plats



Det sista kan enkelt åstadkommas i en enkellänkad lista genom att lägga till tail

Cirkulär lista

Både en enkellänkad lista (och en dubbellänkad lista) kan göras cirkulär genom att länka sista noden till första (och första till sista)

+ Vi kan traversera hela listan från godtyckligt element

+ Riskerar inte att falla av listan

- Måste dock undvika oändliga loopar

Länkad lista i JCF (dubbellänkad)

java.lang.Object

java.util.AbstractCollection<E>

java.util.AbstractList<E>

java.util.AbstractSequentialList<E>

java.util.LinkedList<E>

All Implemented Interfaces:Serializable, Cloneable, Iterable<E>, Collection<E>, Deque<E>, List<E>, Queue<E>

LinkedList<E> Iterator

LinkedList implementerar Iterable och har därmed metoden:

```
public Iterator<E> iterator()
```

Iterator är ett interface med följande metoder:

Method	Behavior
`boolean hasNext()`	Returns `true` if the `next` method returns a value.
`E next()`	Returns the next element. If there are no more elements, throws the `NoSuchElementException`.
`void remove()`	Removes the last element returned by the `next` method.

JCF Iterator

En iterator traverserar alla element med O(n)

```
Iterator<Integer> iter = aList.iterator();
while (iter.hasNext()) {
  int value = iter.next();
  // Gör något med value
}
```

istället för

```
for(int index=0;index<aList.size();index++){
  int value = aList.get(index);
  //Gör något med value
}
```

som blir $O(n^2)$

Finns också enhanced for statement som använder iteratorn:

```
for(int value: aList){
   //Gör något med value
}
```

Exempel - remove

Du måste anropa next före varje remove

```
public static void removeDivisibleBy(LinkedList<Integer> aList, int div) {

  Iterator<Integer> iter = aList.iterator();

  while (iter.hasNext()) {
    int nextInt = iter.next();
    if (nextInt % div == 0) {
      iter.remove();
    }
  }

}
```

ListIterator

- Listiterator<E> interfacet ärver från Iterator<E>
- Tillhandahåller utökad funktionalitet
- Bla LinkedList tillhandahåller en ListIterator
- Viktigaste förbättringarna:
  - add
  - traversera listan i bägge riktningarna

- För att få en sådan anropa listIterator(index)
- Se Java API för alla metoder